Deep Learning - project 1. Convolutional Neural Networks in image classification.

Experimentation report

authors: Barbara Seweryn, Michał Wietecki
Warsaw University of Technology

May 5, 2026

Contents

1 Abstract

The objective of this project is to compare and analyze the performance of different neural network architectures (convolutional neural network and transformers) and their pretrained versions in the speech classification problem based on the Tensorflow Speech Commands Datasets dataset. Experiments with different hyperparameters and strategies are conducted and their influence on the model results is investigated. Overall, the CNN model achieves better and more stable results than both versions of the transformer. The best-performing CNN configuration (n_filters=128, dropout=0.3) achieves validation F1 of 0.961 on the full 12-class dataset, substantially outperforming the transformer trained from scratch and pretrained transformer (for both F1=0.925). CNN models demonstrate superior training stability with lower overfitting gaps (1.54–4.74 pp) compared to transformers (4.63–7.26 pp). However, all models face significant challenges with class imbalance, where the dominant `unknown` class (61.1% of validation set) causes systematic misclassification of command classes. While strategies such as weighted sampling and two-step cascade architectures provided some improvements in command classification, they led to a performance collapse in the `silence` class, which was a category that had been recognized with near-perfect recall in previous stages.

2 Description of the research problem

This project focuses on speech classification with the use of neural networks. The main objective of the project is to experiment with different neural network architectures and their hyperparameter and evaluate their performance.

The Speech Classification dataset includes 65,000 one-second long recordings of 30 short words, by thousands of different people. The networks investigated in our project are custom convolutional neural network, a transformer trained from scratch and a pretrained transformer. The networks are described in details in Section 3.

3 Theoretical introduction

3.1 Network architectures

In the project, we test a custom convolutional neural network (CNN), a transformer trained from scratch, and a pretrained transformer. We test and evaluate these models with different values of hyperparameters to analyze and understand the behavior of the models. Below, we present an overview of the models and their hyperparameters on which our experiments were focused.

3.1.1 CNN

CNNs are a class of deep learning models, which used mainly in computer vision, but can also be utilized in audio processing. One of the main building blocks of CNNa are convolutional layers. They perform key mathematical operations known as convolution, which are responsible for recognizing patterns. The convolutional layers perform feature extraction and create feature maps. They are followed by pooling layers, which reduce the dimension of feature maps, and fully connected layers. The latter acts as a traditional neural network, which weighs the influence of recognized patterns on the final label.

In speech classification tasks, the audio signal is typically transformed into a time–frequency representation (e.g., a spectrogram), which a CNN processes similarly to an image, learning local patterns across both dimensions. Convolutional layers apply multiple learnable filters that slide over the input and respond to specific acoustic patterns producing feature maps that highlight where these patterns occur. Pooling operations improve robustness to small shifts in time and frequency, while deeper layers combine the outputs of many filters into higher-level representations corresponding to target classes (e.g., words).

Our custom CNN consists of three convolutional blocks in which the number of filters increases progressively (from `n_filters` to `4n_filters`, where `n_filters` is a model hyperparameter). Each block uses ReLU activation function and in the first two cases—downsampling via max pooling, which reduces spatial dimensions. This is followed by global average pooling (AdaptiveAvgPool2d) that compresses the feature maps into a vector, which is then regularized with dropout and passed to a linear layer for final classification. The implementation of this model architecture is shown in Listing 1.

```
self.conv = nn.Sequential(
    nn.Conv2d(1, n_filters, 3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),

    nn.Conv2d(n_filters, n_filters*2, 3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),

    nn.Conv2d(n_filters*2, n_filters*4, 3, padding=1),
    nn.ReLU(),

    nn.AdaptiveAvgPool2d((1, 1))
)

self.fc = nn.Sequential(
    nn.Dropout(drop_rate),
    nn.Linear(n_filters*4, num_classes)
)
```

Listing 1: CNN architecture

3.1.2 Transformers

Transformers are a class of neural network architectures based on self-attention mechanisms, originally introduced in [2]. They process input sequences in parallel and learn their context by assigning attention weights between all elements of the input. This makes them very effective for processing text and audio, which can have complex temporal patterns.

When processing audio the input is divided into smaller segments, embedded into a higher-dimensional space, and processed through multiple layers of self-attention and feed-forward networks. This allows the model to learn local acoustic features and global temporal structure of the input, which is important for distinguishing between different words in the speech samples.

The structure of the first transformer we test in our experiments in presented in Listing 2. As it can be seen the `pretrained` parameter has been set to *False*, since we want to investigate the behavior of a transformer that is trained from scratch, only on the Speech Classification dataset. The `depth`, `num_heads` and `drop_rate` hyperparameter value was modified in different experiments to observe

model behavior. The depth hyperparamter represents the number of transformer encoder blocks, while `num_heads` controls the number of attention heads per block. The `drop_rate` hyperparameter represents the dropout applied within the model (randomly "dropping" (setting to zero) a fraction of neurons' outputs with a given probability) to reduce overfitting.

```
timm.create_model(
        'vit_tiny_patch16_224',
        pretrained=False,
        in_chans=1,
        num_classes=num_classes,
        drop_rate=drop_rate,
        depth=n_layers,
        num_heads=n_heads,
        img_size=(64, 32),
        patch_size=patch_size
    )
```

Listing 2: Architecture of the transformer trained from scratch

The second transformer we decided to test was a pretrained transformer. For this model the architecture was analogous to Listing 2, but with the `pretrained` parameter set to *True* and set values of `depth`, `num_heads` and `drop_rate` hyperparameters. Therefore the model was already trained on a large-scale image dataset, and we adapt it to the speech classification task. For this model we test different fine-tuning strategies: *["freeze", "partial", "none"]*. The *"freeze"* strategy keeps all initial model parameters frozen except for the classification head. In this scenario the initial transformer acts as a fixed feature extractor and it trains fast, with low risk of overfitting. The *"partial"* strategy keeps most of the model is frozen, except the last transformer block and the classification head. It balances stability (from pretrained weights) with flexibility to the specific task. The last approach, `strategy = "none"`, represents full fine-tuning, meaning all parameters are trainable. The entire transformer is updated during training, which results in maximum flexibility and adaptation to speech data. However it also requires much higher computational cost and has a greater risk of overfitting, especially with small datasets.

4 Conducted experiments

4.1 Description of experimentation process

Our experimentation was focused on 3 neural networks: our custom convolutional neural network, a transformer trained from scratch and a pretained transformer. Before training the pretrained transformer on our dataset it fetched the pretrained model weights and starts the training process from said weights.

The experimentation process was divided into 5 stages. The outline of these stages is presented in Table 1. The first stage was focused on training baseline models, which are used for reference evaluation of other models. The second stage focused on testing models with different hyperparameter values. To save computational resources the models in the first two stages were trained on a subset of the dataset. The subset consisted of instances of six classes (*"yes", "no", "up", "down", "left", "right"*). In the third stage we investigated their influence on model performance and chose the best performing or most promising models for further experimentation. In the forth stage we trained the chosen models

on the entire set of classes (*"yes", "no", "up", "down", "left", "right", "on", "off", "stop", "go", "silence", "unknown"*) and investigated their performance. The fifth stage was specifically focused on distinguishing the "silence" and "unknown" classes from the rest, since these classes introduced chaos in the models. In this stage we tested different approaches to distinguishing those classes.

Stage	**Subject**	**Details**
1	Baseline training	Custom CNN, transformer and pretrained transformer training on the reduced dataset and evaluation. These models are later used as the baseline for the second and third stage.
2	Hyperparameter testing	Investigation of influence of different hyperparameters on the model performance. Tested hyperparameters: • CNN - n_filters: [16, 32, 64, 128, 256, 512], dropout: [0.0, 0.1, 0.2, 0.3, 0.5], • Transformer - n_layers: [2, 4, 8],n_heads: [4, 8, 16], dropout: [0.0, 0.1, 0.2, 0.3, 0.5], • Pretrained Transformer - strategy: ["freeze", "partial", "none"].
3	Model comparison and investigation	Investigation of influence of the hyperparameters on the model performance and choosing the best models.
4	Training the chosen models on full dataset	Training the chosen models on the dataset containing all the classes to check if they perform similarly when handing more complex data.
5	Addressing problems with "unknown" and "silence" classes	Testing different strategies for addressing the trouble those classes: • weighted sampling, • two-step model.

Table 1: Experimentation process outline

4.2 Training environment

All computations were performed using GPU T4 x2 available in Kaggle. Kaggle offers 30 hours of free GPU access per user per week, therefore in the planning process we needed to focus on computational limitations. All models were trained using the same loss function (Multiclass Cross Entropy), optimizer (ADAM), batch size (32), number of epochs (25), learning rate (0.0005) and scheduler (ReduceLROnPlateau) to ensure a sensible comparison environment. To ensure experiment reproducibility, a random seed was fixed.

All models were evaluated after each epoch. During evaluation training accuracy, validation accuracy, training F1 score and validation F1 score were calculated and saved.

5 Results

This section contains results of each experimentation stage in the order they were conducted.

5.1 Stage 1: Baseline Training

The primary objective of this stage was to establish baseline performance for all three models trained with default hyperparameters on the reduced six-class dataset (*yes, no, up, down, left, right*). The key performance metrics of the models trained in this stage are presented in Table 2.

Table 2: Stage 1 baseline results on the reduced dataset (after 25 epochs).

Model	Best Val F1	Best Epoch	Final Val F1
CNN (baseline)	0.9562	22	0.9553
Transformer (scratch)	0.8677	24	0.8665
Transformer (pretrained)	0.9040	15	0.8969

CNN outperforms Transformers. The CNN achieved the highest validation F1 of 0.956, higher than both the scratch-trained transformer (0.868) and the pretrained transformer (0.904). While unexpected given the recent dominance of transformers in various domains, this outcome may be linked to several factors specific to our experimental environment. First, the dataset is relatively small (six classes, 2350-2380 one second recordings per class), and transformers are known to require substantially more data to leverage their global self-attention mechanism effectively [1]. Secondly, the input are represented as log-mel spectrograms. They are two-dimensional time–frequency images whose local structure is precisely what convolutional filters are designed to capture. Convolutional layers search for local acoustic details using sliding kernels. Because of pooling, the model identifies these patterns even if their location changes. This natural alignment with the structure of sound makes CNNs highly effective for spectrogram-based tasks. Transformers, by contrast, have no such built-in locality bias. They learn all spatial relationships from scratch or from an unrelated pre-training task. Finally, our custom CNN has approximately ∼180,000 parameters at `n_filters`=64, which is lean enough to converge reliably within 25 epochs, whereas the transformer architectures are considerably larger and may not have enough information to recognize global patterns.

Overfitting in Transformer models. Validation curves for all three architectures are presented in Figure 1. For both transformer models, validation loss begins to rise after 10-15 epochs while validation accuracy continues to increase or starts to converge. This is a clear sign of overfitting. The CNN model, by contrast, maintains a decreasing loss function curve all the way to 25th epoch.

5.2 Stage 2: Hyperparameter Tuning

In Stage 2 we investigated the effect of architecture-specific hyperparameters on model performance. All configurations were trained with the same fixed settings as specified in 4.2. The searched grids were:

- **CNN** (30 configurations):
  `n_filters` $\in \{16, 32, 64, 128, 256, 512\}$;
  `dropout` $\in \{0.0, 0.1, 0.2, 0.3, 0.5\}$.
- **Transformer (scratch)** (45 configurations):
  `n_layers` $\in \{2, 4, 8\}$;
  `n_heads` $\in \{4, 8, 16\}$;
  `dropout` $\in \{0.0, 0.1, 0.2, 0.3, 0.5\}$.

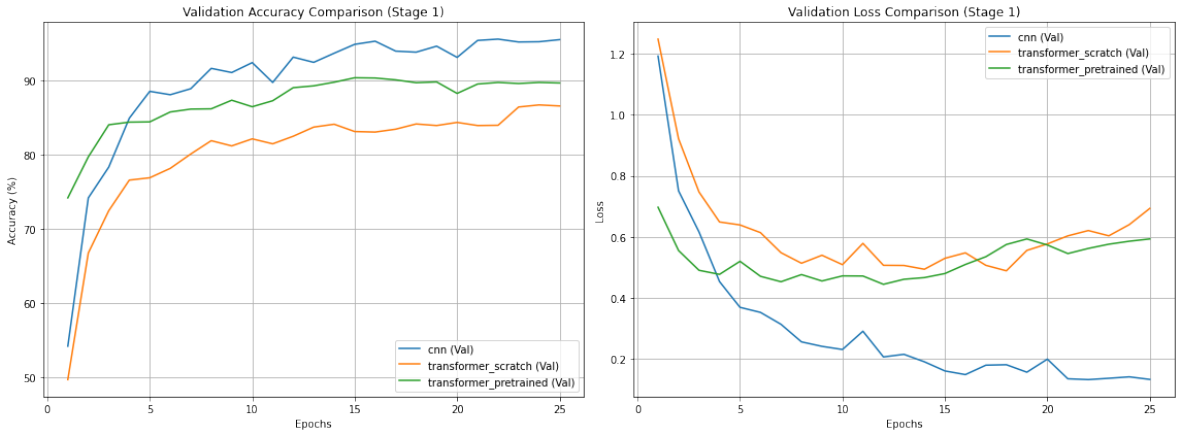

Figure 1: Validation accuracy (left) and validation loss (right) for the three baseline architectures over 25 epochs. Both transformer models show a rising validation loss after 10-15 epoch, indicating overfitting.

- **Pretrained Transformer** (3 configurations):
  fine-tuning strategy $\in$ {*freeze*, *partial*, *none*}.

Pretrained Transformer: fine-tuning strategy. The three fine-tuning strategies differ in how much of the pretrained network is allowed to update during training: *freeze* locks all transformer weights and trains only the classification head, *partial* unfreezes the top blocks and the head, *none* performs full fine-tuning from the pretrained initialization. The results of this experimentation are shown in Table 3.

Table 3: Best validation F1 for each pretrained transformer fine-tuning strategy.

Strategy	**Best Val F1**	**Best Epoch**
None (full fine-tuning)	**0.9156**	24
Partial	0.8242	16
Freeze	0.4627	25

Full fine-tuning achieves the best result (0.916), significantly outperforming *partial* (0.824) and *freeze* (0.463). The poor performance of the *freeze* strategy is due to the fact that feature extractor remains fixed and never adapts to the target domain, and the a linear head itself cannot bridge this gap. The accuracy curves, presented in Figure 2), show similar training performance for both *none* and *partial* strategies, as both can fit the training set. However, the *none* strategy achieves higher validation accuracy because unfreezing all layers gives the model the physical capacity to adapt its entire feature extraction process to the target domain. While *partial* is restricted to adapting only the final blocks, the *none* strategy optimizes all parameters, allowing it to learn more generalizable patterns rather than just memorizing training samples. This increased flexibility results in a model that performs significantly better on the validation dataset, however it did require more computational effort.

Distribution of results: CNN vs. Transformer. In Figure 3 the distribution of best validation F1 across all configurations for CNN and non-pretrained transformer is presented. The results have visibly different ranges. The median of CNN configurations is around 0.95. All 45 transformer

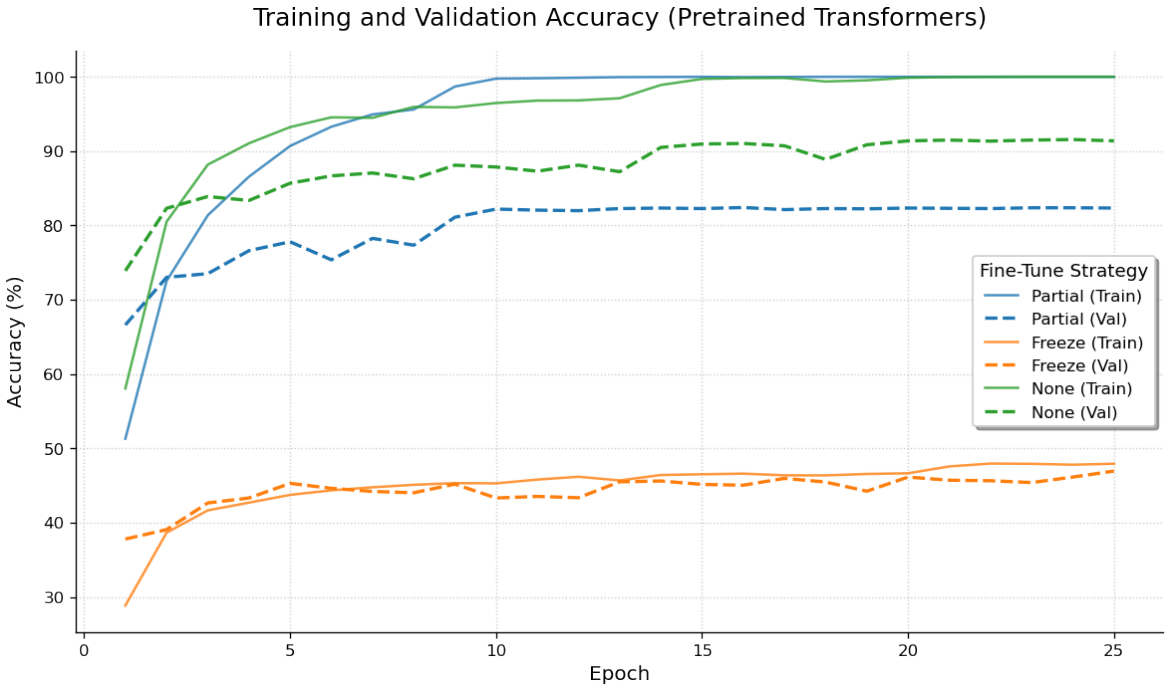

Figure 2: Training and validation accuracy for the three pretrained transformer fine-tuning strategies over 25 epochs.

configurations fall within the narrow range of 0.827–0.876. This saturation of transformer performance regardless of depth, head count, or dropout may suggest that the architecture has reached a task-specific limit under the given training budget. Additional capacity provides no advantage, likely because 25 epochs are insufficient to exploit deeper attention hierarchies at this dataset size. The patterns in mel-spectrograms are better captured by convolutional inductive biases than by global self-attention.

CNN: influence of `n_filters` and `dropout`. As visible in Figure 5, for `n_filters` ≤ 128, the maximal validation F1 measures are very high. Increasing the number of filters from 16 to 32 and 64 results in 1-2 pp. increase of F1 performance. When regularization in the form of dropout is turned off completely, lowering the number of filters from 32/64 to 128 decreases the metric value by 3 pp.; however, for the other configurations the level is comparable. For the dropout set at 0.3 we scored the highest value of F1 = 0.966. Increasing `n_filters` > 128 introduces significant anomalies, ranging from total model collapse to severely degraded performance. While optimization instability and early gradient issues often disrupt training, Figure 4 reveals that some models simply fail to converge. Instead of plateauing, their accuracy grows linearly through epoch 25, proving these larger architectures are severely under-trained and inefficient compared to smaller models.

Regarding dropout, within the stable region (`n_filters` ≤ 128) the right columns of the overfitting-gap heatmap (Figure 5) show a clear trend: higher dropout slightly reduces the gap between training and validation F1. However, the effect on absolute validation performance is mild — configurations with dropout $\in \{0.1, 0.2, 0.3\}$ all achieve ~ 0.963–0.966, indicating that the task is not difficult enough for these model sizes to require heavy regularization.

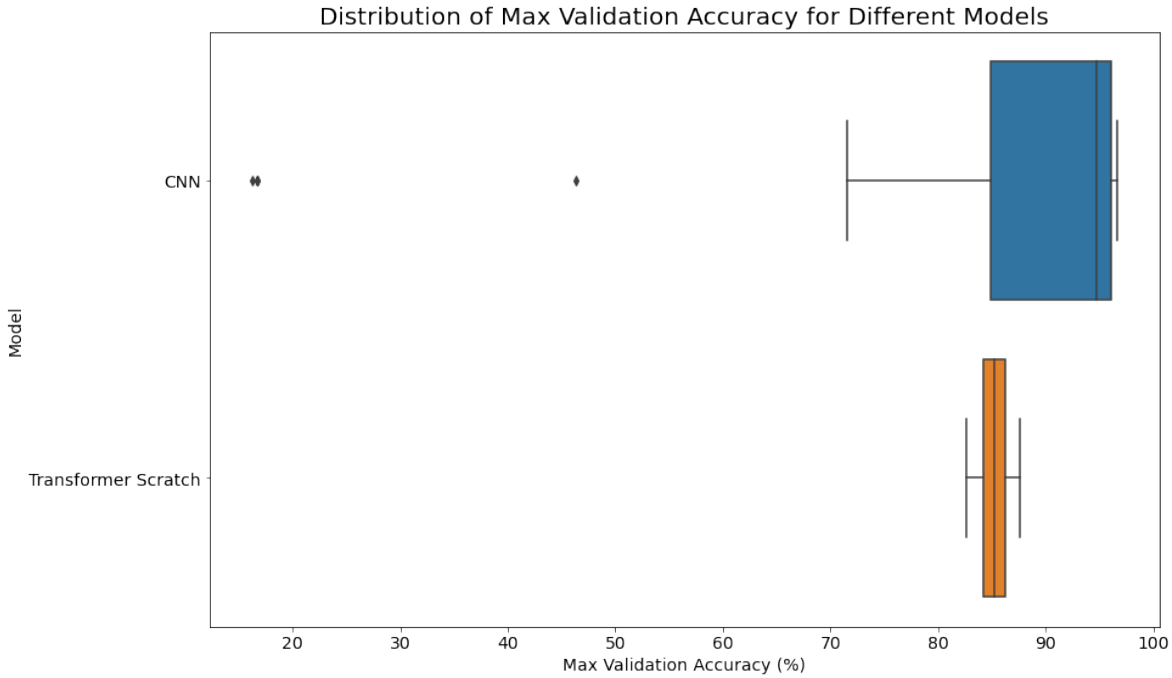

Figure 3: Distribution of best validation F1 across all Stage 2 configurations, grouped by architecture. Transformer results are tightly clustered around 0.85; CNN results peak near 0.96, but the variation is higher.

Figure 4: Training and validation accuracy for CNN configurations of `dropout` $= 0.2$ and `n_filters` > 128, illustrating linear growth and incomplete convergence that fails to saturate within the 25-epoch budget.

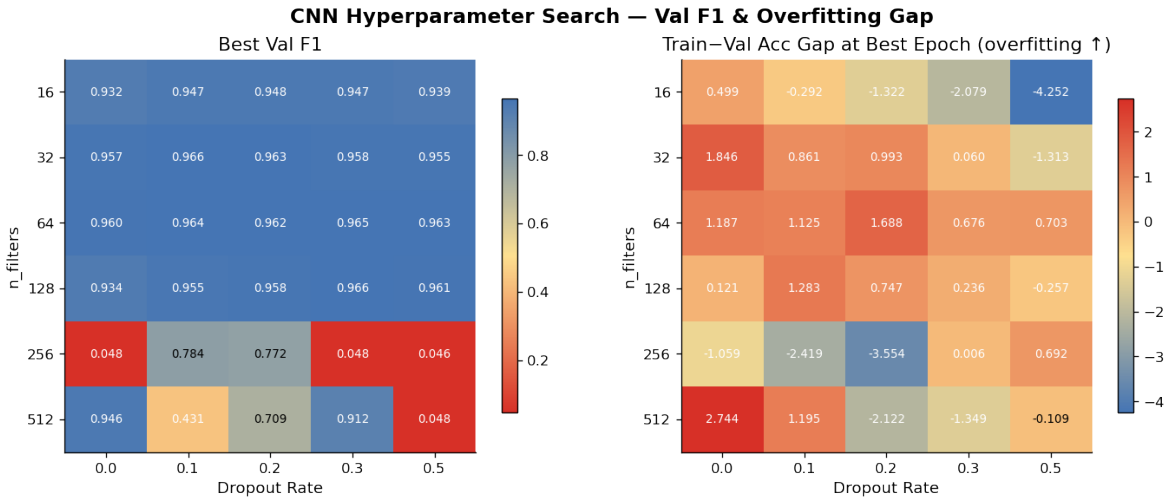

Figure 5: Left: best validation F1 heatmap for all CNN configurations. Right: overfitting gap (train accuracy − val accuracy from the epoch with highest F1) per configuration. Higher dropout (right columns) slightly reduces the overfitting gap.

Convergence speed. Figure 6 plots the exact epoch at which each model first reaches 95% of its best validation F1 against that best value. CNN configurations achieving validation F1 > 0.9 typically reach this threshold between epochs 5 and 15; the best-performing CNN models often converge as early as epoch 5–8. All transformer configurations show a similar 5–15 epoch range. The few CNN outliers appearing far to the left (epoch ≤ 4) at close-to-zero F1 values correspond to the collapsed `n_filters` = 256/512 models, while several dots on the right ilutrate models that did not manage to converge in time (like configurations in Figure 4).

Training stability: per-class recall. Figure 7 shows per-class validation recall over training for CNN and Transformer (scratch) configuration from Stage 2 that achieved highest maximum F1 values. The CNN's per-class curves converge to stable plateaus within the first 10 epochs and remain very smooth. The Transformer's curves, by contrast, exhibit high-amplitude oscillations that persist throughout all 25 epochs. This instability is a known characteristic of training transformers on small datasets [1]. It further reinforces the CNN's practical advantage in small training examples environments: not only does it achieve higher validation F1, but it does so more reliably and with less training noise. Stopping training at a specific epoch should produce similar results across adjacent epochs.

Confusion analysis. Figure 8 shows that "no" and "down" represent the most frequent misclassifications, as they share significant mel-spectrogram characteristics. Both words feature a similar low-frequency onset and a falling vowel trajectory. Key subtle differences may be compressed or lost at our level of input feature resolution, making the two classes difficult to separate consistently for both convolutional filters and attention heads.

5.3 Stage 3: Model Selection for Further Training

Based on the findings of Stages 1 and 2, we selected models for full-dataset training in Stage 4 following a consistent strategy: for each architecture we chose the best-performing configuration as the primary candidate, complemented by two additional variants differing in capacity or regularisation strength, in order to observe how these factors interact with the increased task complexity.

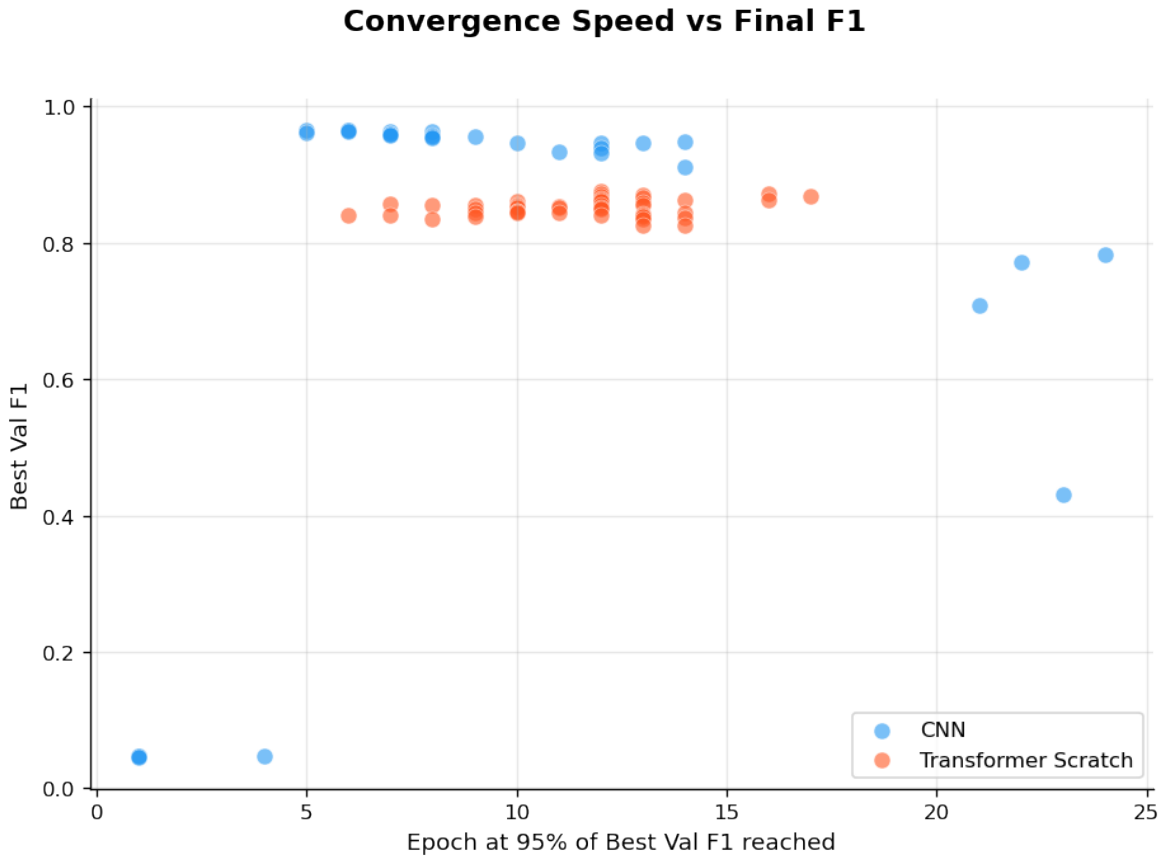

Figure 6: Epoch at which each model first reaches 95% of its best validation F1, plotted against that best F1. Most well-performing CNN models converge within epochs 5–15; the best CNNs reach this threshold as early as epoch 5-8.

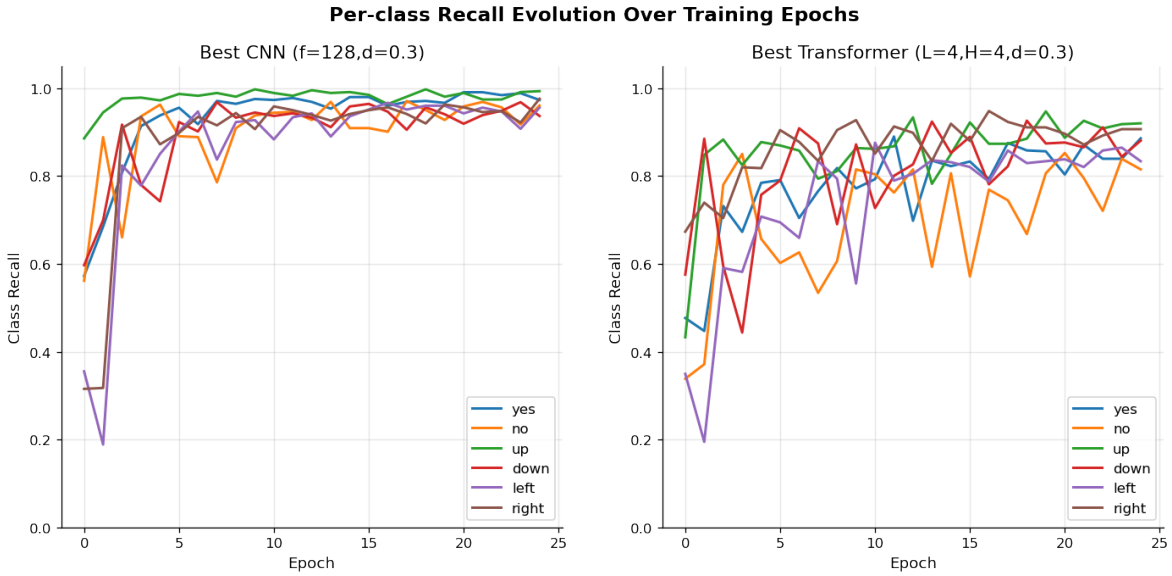

Figure 7: Per-class validation recall over 25 training epochs for the best CNN (left) and best Transformer scratch (right) configuration from Stage 2. CNN recall curves stabilize rapidly; Transformer curves exhibit persistent oscillations.

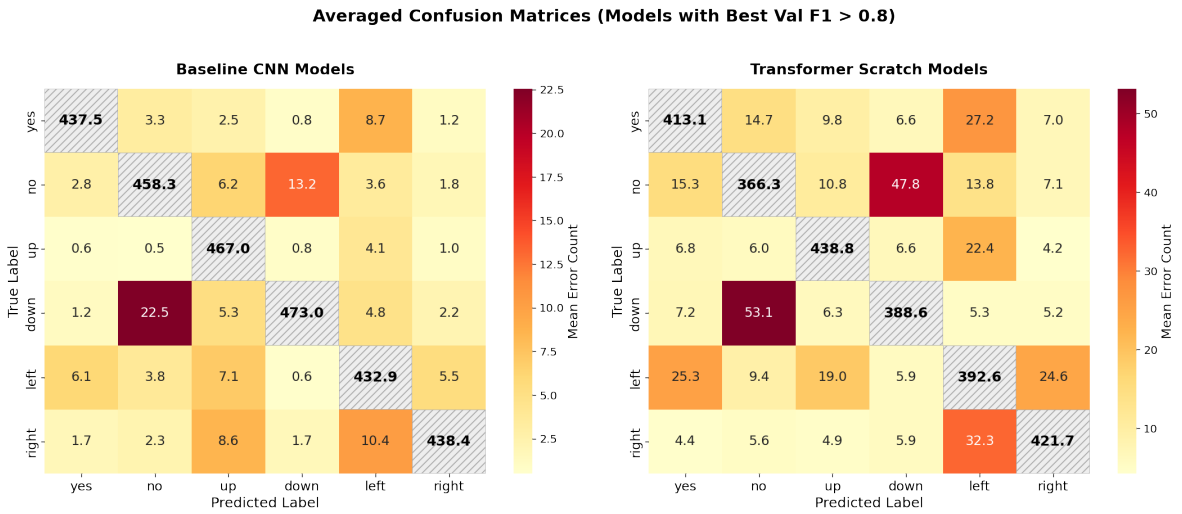

Figure 8: Averaged confusion matrices for the best CNN (left) and best Transformer scratch (right) configurations. The most significant off-diagonal entries correspond to "no" ↔ "down" confusions in both architectures.

CNN. Three configurations were selected:

- **CNN-1** (*primary*): `n_filters`=128, `dropout`=0.3. Best overall validation F1 (0.966), small overfitting gap (0.24%), and rapid convergence (plateau reached at epoch 6).
- **CNN-2** (*large, no regularisation*): `n_filters`=512, `dropout`=0.0. Represents the highest-capacity configuration with the highest F1 reached; included to test whether a larger network benefits from the richer 12-class signal despite its training instability on the reduced dataset.
- **CNN-3** (*small, heavy regularisation*): `n_filters`=32, `dropout`=0.5. A compact model with strong dropout; included as a lower-complexity reference to assess the trade-off between regularization and capacity at scale.

Transformer (scratch). Three configurations were selected:

- **Transformer-1** (*primary*): `n_layers`=4, `n_heads`=4, `dropout`=0.3. Highest validation F1 among all scratch transformer configurations (0.876).
- **Transformer-2** (*larger network*): `n_layers`=8, `n_heads`=4, `dropout`=0.2. One of the top-performing configurations (0.873); included to evaluate whether additional depth helps on the full dataset.
- **Transformer-3** (*primary architecture, no regularization*): same architecture as the primary candidate but without `dropout` (0.872); included to isolate the effect of regularization.

Pretrained Transformer. Only the *none* (full fine-tuning) and *partial* strategies were carried forward, as the *freeze* strategy achieved F1 = 0.463 on the reduced dataset, which was far below the other configurations and showing no signs of improvement over 25 epochs.

5.4 Stage 4: Training on the Full Dataset

The objective of this stage was to evaluate whether the models selected in Stage 3 retain their performance characteristics when trained on the full 12-class dataset. Compared to the reduced six-class setup, the task is substantially more complex: the number of target classes doubles, and two structurally atypical classes are introduced — `silence` (recordings containing no speech) and `unknown` (a catch-all category aggregating all words outside the ten core commands). Crucially, the dataset is heavily imbalanced: each of the ten command classes and `silence` represents approximately 3.5% of the validation set, while `unknown` accounts for 61.1%. All training settings were kept identical to those described in 4.2.

Overall results. Table 4 summarizes the key performance metrics for all models trained in this stage. **CNN-1** remains the best-performing model overall, achieving the best validation F1 of 0.961 at epoch 19, which is almost identical to its Stage 2 result of 0.966 on the reduced dataset, confirming that this configuration is reliably generalized to a more complex task. **CNN-3** (small, heavily regularized) achieves a competitive 0.946, with the smallest overfitting gap of all models (−1.31 pp, (validation accuracy slightly exceeds training accuracy) shows that dropout of 0.5 acts as a strong regularizer even at this scale). **CNN-2** (large, no regularization) drops to 0.937 and shows a 4.74 pp overfitting gap, consistent with the training instability observed for this configuration in Stage 2.

Among transformers trained from scratch, all three configurations improve over their Stage 2 results. **Transformer-1** rises from 0.876 to 0.925, **Transformer-2** from 0.873 to 0.920, and **Transformer-3** from 0.872 to 0.919. This improvement can be attributed to the richer training signal provided by the full dataset. More classes show the model a greater variety of acoustic patterns. The `unknown` class serves as a representative sample background of commands, which forces the model to refine its decision boundaries. By seeing many different words from the `unknown` class, the attention mechanism may start to ignore generic speech patterns and focus only on the unique features that define each specific command. Despite this gain, all transformer configurations exhibit considerably larger overfitting gaps (6.5–6.7 pp) than **CNN-1**, and **Transformer-1** and **Transformer-2** have not converged by epoch 25, as the best F1 was in epoch 25. This suggests that they would benefit from longer training time.

The pretrained transformer with no fine-tuning strategy chosen achieves F1 of 0.925. This is comparable to the scratch transformers, but with a larger overfitting gap of 7.26 pp. The model also scored a near-perfect training accuracy of 99.9%, indicating that it memorized the training set rather than generalizing seen patterns. The *partial* strategy performs a lot worse (0.813), with a huge overfitting gap of 18.09 pp, which is the largest of all configurations. This confirms the pattern observed in Stage 2: restricting tuning to only the final blocks and classification head is insufficient when the target domain of speech spectrograms, differs greatly from the domain, on which the model had been pretrained (natural images).

Class imbalance and the `unknown` class problem. Although aggregate F1 scores appear high, a per-class recall breakdown reveals a more detailed picture. Figure 9 presents recall for three categories: ten core command classes, `silence`, and `unknown`. Across all models, `silence` is classified with perfect recall of 1.0 for almost all models. `Unknown` class also has high recall values. The command classes, by contrast, are consistently less well-recalled, with averages ranging from 0.619 (Pretrained *partial*) to 0.909 (**CNN-1**). The reason for this pattern is directly tied to class imbalance. With `unknown` comprising 61.1% of the validation set, the class represents an extremely broad distribution of acoustic patterns. Consequently, when a model is really uncertain about a specific command, the observation

Table 4: Comparison of performance between Stage 2 (6 classes) and Stage 4 (12 classes).

	Stage 2	Stage 4 (Full 12-class dataset)				
Model	**Best F1**	**Best F1**	**Epoch**	**Final F1**	**Train Acc**	**Gap**
CNN-1	0.966	0.9614	19	0.9609	97.72%	+1.54 pp
CNN-2	0.946	0.9365	24	0.9357	98.42%	+4.74 pp
CNN-3	0.954	0.9461	22	0.9332	93.37%	−1.31 pp
Transformer-1	0.876	0.9252	25	0.9252	99.10%	+6.47 pp
Transformer-2	0.873	0.9200	25	0.9200	96.75%	+4.63 pp
Transformer-3	0.872	0.9188	23	0.9163	98.58%	+6.69 pp
Pretrained (`none`)	0.916	0.9250	22	0.9242	99.88%	+7.26 pp
Pretrained (`partial`)	0.824	0.8128	24	0.8123	99.97%	+18.09 pp

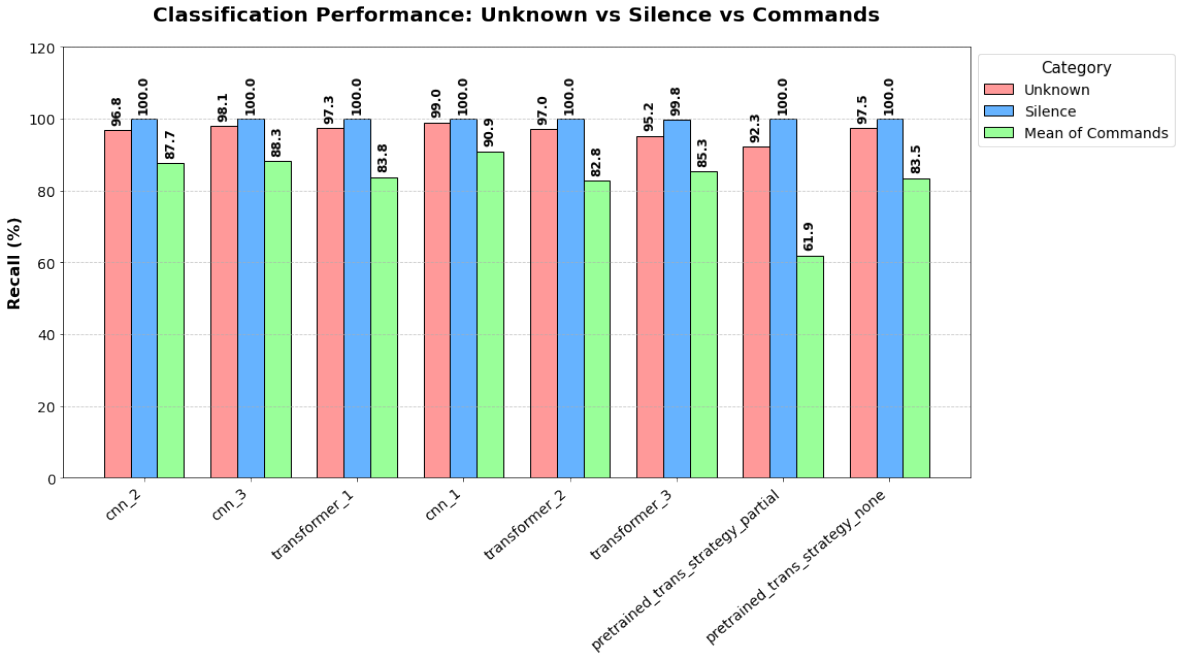

Figure 9: Validation recall for `unknown`, `silence`, and target command classes (mean) across all evaluated models in Stage 4.

is statistically much more likely to its features with diverse samples from the `unknown` class. This was confirmed by the confusion matrix analysis. For **CNN-1**, between 4% and 11.8% of samples from each command class are misclassified as `unknown`, with the most affected classes being `on` (11.8%) and `right` (9.9%). The pattern is more pronounced for the scratch transformers, where misclassification into `unknown` reaches up to 14.8% (`on` class for **Transformer-1**). Misclassification into `silence` is negligible across all models. The averaged confusion matrix for **CNN-1** and **Transformer-1** is presented in Figure 10.

The dataset structure motivates two approaches investigated in Stage 5. The first is a two-step model: a classifier (*filter*) first distinguishes between `silence`, `unknown`, and commands, and only samples classified as commands are passed to a second model (*specialist*) trained exclusively on the ten command classes. The second approach is to apply weighted sampling during training, counteracting the dominance of the `unknown` class by oversampling command classes, forcing the model to learn more balanced decision boundaries.

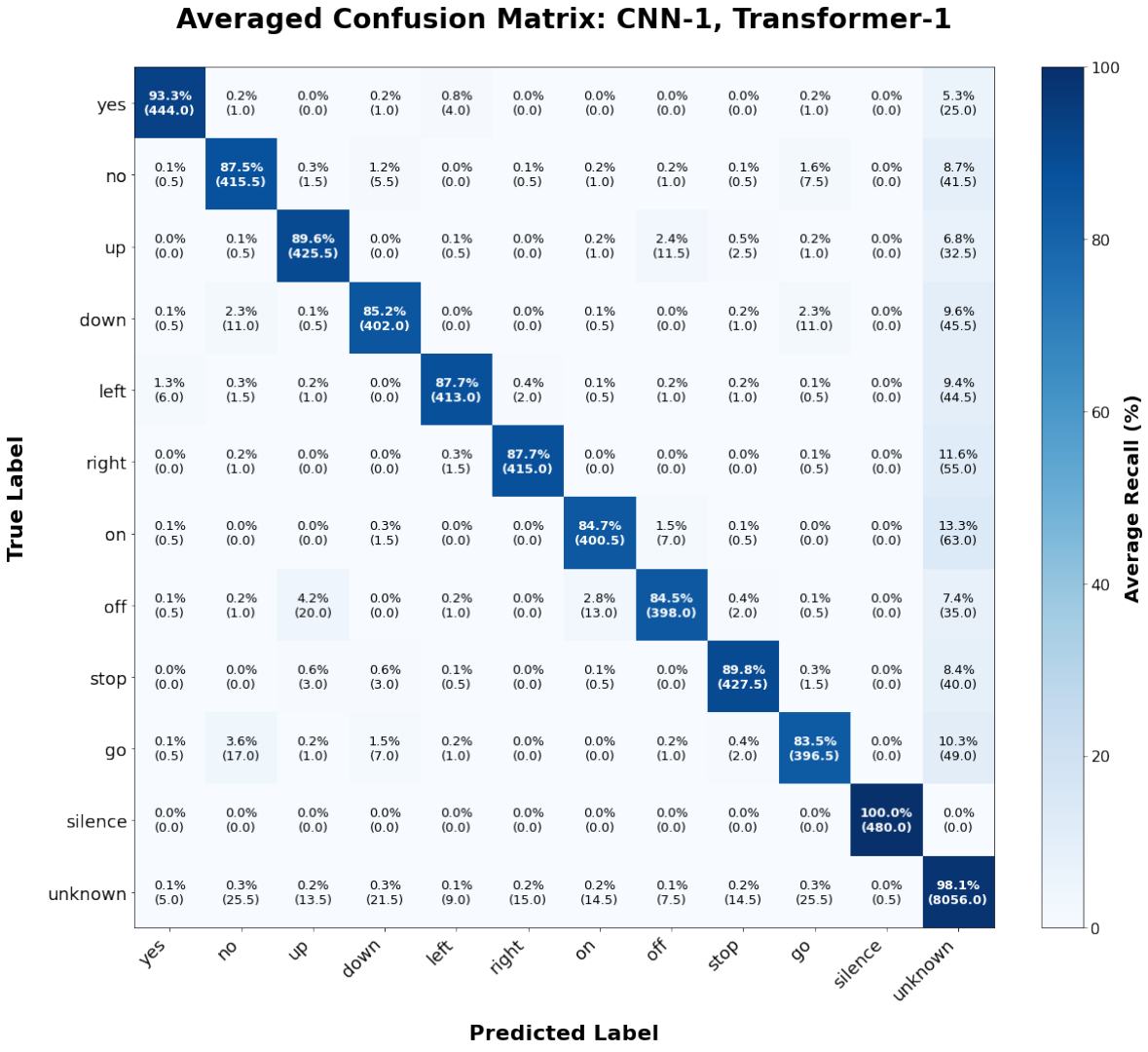

Figure 10: Averaged confusion matrix for the primary Stage 4 models (**CNN-1** and **Transformer-1**). Each cell represents the mean classification result, displaying the row-normalized values as primary value (in this setting diagonal represents recall) and the average absolute sample count in parentheses below. The matrix highlights the systematic misclassification of command classes into the `unknown` category.

5.5 Stage 5: Addressing the `unknown` Class Problem

Stage 4 revealed a systematic bias in all models. Command classes were consistently misclassified as `unknown`, which was a consequence of the severe class imbalance.

Stage 5 investigated two strategies to mitigate this problem, applied to the best-performing configurations of different from Stage 4: **CNN-1** and **Transformer-1**.

Strategy 1: Weighted sampling. The first approach rebalanced the training distribution by oversampling commands and `silence` classes relative to `unknown`, forcing models to learn more balanced decision boundaries without modifying the architectures.

CNN-1 and **Transformer-1** were retrained with weighted sampling using the same hyperparameters as Stage 4. Results are shown in Table 5.

Table 5: Weighted sampling results compared to Stage 4 baselines (best validation F1).

Model	Stage 4 F1	Sampling F1	Avg. Command Recall	Silence Recall
CNN-1 (weighted)	0.9614	0.9017	0.932	0.015
Transformer-1 (weighted)	0.9252	0.8689	0.842	0.193

Weighted sampling produced a significant trade-off that ultimately degraded overall performance. The mean recall across the ten command classes increased noticeably for both models:

- **CNN-1**: from 0.909 to 0.932;
- **Transformer-1** from 0.838 to 0.842.

However, the F1 dropped substantially:

- **CNN-1**: from 0.961 to 0.902;
- **Transformer-1**: 0.925 to 0.869

The cause is how models now treated class `silence`. Its recall collapsed to near zero for the **CNN** (1.5%) and remained very low for the **Transformer** (19.3%). Oversampling commands relative to `unknown` created a training distribution in which models stopped predicting a previously structurally distinct class `silence`. Since `unknown` recall remained high (94% for **CNN**, 88% for after resampling), after resampling the primary bottleneck happened to be `silence`, not the command classes.

Strategy 2: Two-step cascade. The second approach divides the problem into two specialized models. A *filter* model first classifies each input into one of three categories: `silence`, `unknown`, or *command* (any of the ten core words). Inputs predicted as *command* are then passed to a *specialist* model trained exclusively on the ten command classes, which produces the final label. This architecture was motivated by the observation from Stage 4. We hypothesized that a two-step model could distinguish between the `unknown` class and the joint command classes more effectively than when they are treated separately. Two cascade pipelines were evaluated: one using **CNN-1** models for both stages (**Cascade-1**) and one using **Transformer-1** models (**Cascade-2**). Results are presented in Table 6.

The two cascades produce dramatically different outcomes. **Cascade-1** achieves F1 of 0.922, comparable to the Stage 4 **CNN-1** result (0.961), while substantially improving command-class recall relative to Stage 4. The *filter* successfully finds most `unknown` inputs before the *specialist*. Thus, the

Table 6: Two-step cascade results for CNN and Transformer architectures (Stage 5). Command recall is the mean recall across the ten command classes.

Pipeline	Accuracy	F1	Cmd Recall (mean)	`silence` Recall	`unknown` Recall
Cascade-1	92.70%	0.9215	0.926	0.245	0.957
Cascade-2	71.58%	0.6728	0.355	0.092	0.954

specialist is no longer biased toward `unknown` predictions. `Silence` recall improves to 24.5%, which is still low but considerably better than the weighted-sampling CNN (1.5%).

Cascade-2 failed almost entirely for command classes, with a mean command recall of only 0.355 and a F1 of 0.673. This happened because **Cascade-2** *filter* model routed an overwhelming proportion of command samples into the `unknown` class.

Summary. Table 7 places all Stage 5 results alongside their Stage 4 baselines for direct comparison.

Table 7: Summary of Stage 5 results versus Stage 4 baselines.

Stage	Model / Strategy	Best Val F1	Cmd Recall (mean)
4	CNN-1 (baseline)	0.961	0.909
4	Transformer-1 (baseline)	0.925	0.838
5	CNN-1 + weighted sampling	0.902	0.932
5	Transformer-1 + weighted sampling	0.869	0.842
5	Cascade-1 (CNN)	0.922	0.926
5	Cascade-2 (Transformer)	0.673	0.355

Neither approach improves the F1 metric over the Stage 4 **CNN-1** baseline. Weighted sampling increases command recall but destroys `silence` recall, still resulting in a drop in F1. The CNN cascade achieves a similar trade-off to Stage 4 with better command recall, but the F1 score remains below **CNN-1** due to persistent `silence` misclassification. The transformer cascade is not a viable approach, as the transformer cascade *filter*'s tendency to over-predict `unknown` makes the *specialist* stage meaningless. Overall, **CNN-1** trained jointly on all 12 classes remains the strongest single model, and the `unknown` imbalance problem proves to be not solvable with used techniques and project setting constraints.

6 Conclusion

Experiments across five stages—from baseline training through hyperparameter tuning to strategies for addressing class imbalance—demonstrate that CNNs are better suited for small spectrogram based tasks due to their built-in locality bias. The best CNN configuration achieves F1=0.961 with stable training dynamics, while transformers may require more data and training time to leverage their self-attention mechanism effectively. The severe class imbalance in the dataset (`unknown` class is almost 61.1%) created a fundamental challenge that was not resolved through used sampling strategy or cascade architectures. These findings highlight the importance of considering both architectural fit and dataset characteristics when designing speech classification systems. Future work could focus on other balancing strategies aimed at preserving the high performance of the `silence` class.

7 Source code

The source code is available in our submission on Leon. The trained models are stored in a separate folder in Google Drive, since they are too large to be uploaded to GitHub.

The source code contains two main components: the `main.ipnyb` file and the `/utils/` directory. The `main.ipnyb` file is used to conduct experiments and save trained models. The following files can be found in the `/utils/` directory:

1. config.py - containing the configuration of all of our conducted experiments,
2. models.py - containing the setup of models,
3. data_utils.py - containing data loaders and data augmentations.

To reproduce our results a user needs to:

1. navigate to `utils/config.py` to see the list of all available experiment,
2. locate the experiment they want to run and copy its key,
3. navigate to the `main.py` file, run all imports and functions,
4. run `run_experiment`(name of the chosen experiment).

The models produced in the experiment, along with their evaluation metrics after every epoch, are be automatically saved to a directory with the name corresponding to the experiment name.

To conduct a new experiment a user needs to:

1. navigate to `utils/config.py`,
2. add a new experiment instance,
3. run the new experimented, as described above.

8 AI disclosure

In this project AI assistance was briefly used in planning the project structure, report text editing and generating code for plots. All scientific decisions, including the design of experiments, interpretation of results, and the conceptual framework, were developed and executed solely by the authors. The analyses and conclusions presented in this report represent the authors' original work.

References

[1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc.